

OrangeHRM Master Test Plan

An Agile Quality Assurance Framework for HR Management Systems

Document Type: Master Test Plan & Management Blueprint

Date: June 1, 2026

Project Domain: Human Resource Management Systems (HRMS)

Version: v3.0 (Comprehensive Build)

1. Project Objective & Strategic Rationale

The primary objective of this project is to execute a comprehensive, hands-on Software Quality Assurance (SQA) lifecycle using **OrangeHRM** as the target application. Human Resource Management Systems (HRMS) and administrative dashboards involve complex workflows, data management processes, and highly interdependent user interactions. Defects within such systems frequently manifest through User Interface (UI) components, business logic layers, APIs, and underlying databases. Therefore, this project focuses on validating functionality, data integrity, integration behavior, and overall system reliability through a structured quality assurance framework under constrained execution timelines.

2. Project Schedule & Timeline

To ensure strict alignment with delivery timelines and maintain high velocity within a compressed execution window, the QA cycle is scheduled according to the following hard deadlines:

Milestone Stage	Date
Project Start & Strategy Activation Date	April 25th
Project Close & Final Testing Delivery Date	July 10th

3. Strategic Pillars of Testing

To guarantee full functional coverage and zero-regression deployment, the quality engineering strategy is driven by four technical pillars:

3.1 Manual Testing

- **Exploratory Testing:** Identifying unexpected system behaviors, edge cases, and potential defects through unscripted testing techniques.
- **UI/UX Testing:** Verifying interface consistency, visual design, responsiveness, usability, and overall user experience across the application.

- **Rapid changes:** Ensuring system stability and functionality remain intact despite frequent feature updates and ongoing application changes.

3.2 Test Automation

- **Speed of Implementation:** Executing large-scale test suites efficiently, enabling extensive coverage within constrained delivery timelines.
- **Repeated Tests:** Automating recurring test scenarios to ensure consistent validation across multiple releases and system updates.
- **Accuracy:** Reducing manual oversight and execution inconsistencies through standardized, repeatable automation processes.

3.3 API Validation

- **Integration Gateways:** Validating the data contracts and integration between the Frontend presentation layer and Backend services.
- **Logic and Response Verification:** Bypassing the presentation layout to directly test backend endpoint behavior, asserting system response codes (e.g., 200 OK, 400 Bad Request, 500 Internal Server Error) and payload structural safety.

3.4 Database & Data Integrity Testing

- **Data Mutation & CRUD Controls:** Ensuring that all operations (Create, Read, Update, Delete) are accurately reflected in backend tables (e.g., confirming a user newly registered via the UI appears seamlessly with matching fields in the user table).
- **Hidden UI Defects Detection:** Exposing critical visual deceptions where a UI indicates success, but underlying database operations fail or introduce data anomalies.

4. Techniques Manual

The testing execution follows a rigorous, page-by-page modular approach to guarantee that no functional segment is left uncertified:

- **User Story:** Acceptance criteria were established to define clear requirements using a "BDD" approach. Test cases are then designed to extensively cover all expected user flows.
- **Test Case:** Test cases were implemented as user profiles to comprehensively cover all expected user paths.
- **Bug Report:** The identified defects are accurately documented in error reports, accompanied by clear visual evidence, including screen recordings and images, ensuring that the problem can be easily reproduced for development teams.

Reverse Review Loop: As soon as the Manual Team completes the verification of a specific page or sub-project, the entire scope is officially handed over to the Automation Team. This handover undergoes a comprehensive technical review before automation script authoring kicks off.

5. Technical Automation

- **Script Development:** Functional code scripts are explicitly developed for each established Test Case, encompassing the automation of critical **Smoke Testing** execution suites to quickly verify build stability.
- **Modular Function Review:** All custom code functions, locators, and assertions are code-reviewed continuously to optimize execution accuracy and script maintenance.

Reverse Review Loop: As soon as the Automation Team completes the verification of a specific page or sub-project, the entire scope is officially handed over to the Automation Team. This handover undergoes a comprehensive technical review before automation script authoring kicks off.

5.1 Multi-Team Rotation Policy

Rotation Policy Framework: To avoid bias, minimize blind spots, and expand platform familiarity, teams are enforced to switch operational roles later in the lifecycle. This structural switch enables cross-team evaluation, allowing distinct sub-teams to conduct peer reviews on each other's written artifacts, executed cases, and code scripts.

6. Team Distribution Matrix

To optimize parallel workflow execution and ensure precise domain ownership during this compressed QA cycle, tactical responsibilities are distributed among team members as follows:

QA Operational Domain	Assigned Engineering Specialists
Manual Testing	Afram & Abdelrahman
Automation Testing	Aly & Omer
API Validation	Aly
Database	Aly

7. Project Constraints & Risk Protocols

Identified Constraint 1: Severely Restricted Timeline (6-Day Cycle)
The testing window stretching from April 25th to May 1st allocates an exceptionally narrow timeframe for planning, test execution, and comprehensive sign-off, making prioritization paramount.

Identified Constraint 2: Academic & Resource Constraints (Student-Led Team)
Balancing advanced quality tracking requirements with rigorous academic commitments mandates an optimized, high-velocity blueprint.

9.1 Risk Mitigation Protocols

1. **Risk-Based Test Case Prioritization:** Utilizing priority frameworks to ensure high-visibility user actions (PIM, authentication states, core data mutations) are validated first.
2. **Parallelized Workflow Execution:** Decoupling sprints so that the Manual Team's design and review loop runs concurrently with backend API script development and cross-team automation preparation.

8. Core Tool Stack

The engineering and management infrastructure relies on a production-ready, standardized software stack to fulfill planning, execution, and defect analysis requirements:

- **Excel:** Employed for foundational test data modeling, tabular data preparation, and localized test case scripting blueprints.
- **Jira:** Serves as the central Application Lifecycle Management (ALM) system for agile sprint backlogs, requirement tracking, and bug life-cycle management.
- **IntelliJ IDEA:** Utilized as the primary Integrated Development Environment (IDE) for compiling automation testing scripts, building framework test classes, and executing smoke/regression suites.
- **Postman:** Leveraged for isolated backend API functional checks, regression collection execution, and response payload validation.
- **MySQL:** Integrated to conduct raw database queries, ensuring proper data mutation and CRUD verification across systemic layers.

9. Advanced Management

To ensure absolute transparency and progress tracking, the project implements a sophisticated, multi-tiered ecosystem using Jira:

Workflow Tier	Platform/Tool Used	Core Management Operational Objective
Level 1: Sprint & Requirements	Jira Software	Sprint setup and uploading the entire requirements hierarchy, mapping out all active User Stories and Tasks.
Level 2: Test Case Management	Testiny	Authoring and uploading test cases into Testiny, directly linking them to active Jira tracking tickets for real-time coverage and status tracking.
Level 3: Full QA Sync	Testiny + Zephyr in Jira	Deep integration of Testiny with Zephyr inside Jira, seamlessly synchronizing Manual and Automation efforts into a centralized single-pane-of-glass dashboard.

10. Project Scope Overview

The following metrics encapsulate the comprehensive tactical scale and rigorous deliverables accomplished by the QA engineering team during this high-velocity testing block:

XX TOTAL USER STORIES	XX TEST CASES DESIGNED	XX BUGS REPORTED
XX MODULES COVERED	XX API ENDPOINTS	XX AUTOMATION SCRIPTS